

Информатика. Методическое пособие по задаче №13.

Содержание

1	Теоретическое введение	2
1.1	IP-адрес	2
1.2	Маска и адрес подсети	2
1.3	Модуль ipaddress	3
1.3.1	Функция ip_address()	4
1.3.2	Функция ip_network()	4
1.3.3	Получение двоичной записи IP-адреса	6
1.3.4	Принадлежность IP-адреса сети	7
1.3.5	Адрес сети и широковещательный адрес	8
1.3.6	Другие полезные свойства	9
2	Разбор прототипов задач	10
2.1	Поиск байта маски	10
2.2	Поиск наибольшего/наименьшего байта маски	12
2.3	Поиск количества значений маски	14
2.4	Поиск номера компьютера	16
2.5	Поиск количества адресов в сети	18
2.6	Кратность единиц IP-адреса	20
3	Заключение	22

1 Теоретическое введение

Для работы с 13-м номером необходимо изучить следующие понятия:

- Базовые понятия о сетях: IP-адрес, маска сети, адрес сети;
- Как находить адрес сети по IP-адресу и маске;
- Базовые функции для работы с сетями в *Python* из модуля *ipaddress*.

1.1 IP-адрес

Для того, чтобы понять, что такое IP-адрес возьмем реальный адрес человека. Например, дом 156 и квартиру 48. Запишем их последовательно: 15648.

Аналогично для каждого компьютера, который подключен к интернету, существует свой уникальный адрес, который называется **IP-адресом**.

Важно!!!

IP-адрес компьютера состоит из четырех байт (**от 0 до 255**), которые разделяются точками.

1.2 Маска и адрес подсети

Адрес подсети и маска подсети – это термины, связанные с IP-адресами устройств в сети Интернет. Они помогают определить, какие устройства находятся в одной подсети. Но что же это такое?

Вернемся к нашему примеру с адресом человека. Мы записали его как 15648. Однако, из такой записи очень трудно понять где номер дома, а где номер квартиры. Именно для этого и существует понятие маски.

При этом важно отметить, что при работе с IP-адресом, адресом сети и маской сети используется **двоичная система счисления**.

Важно!!!

Маска определяет, какая часть IP-адреса указывает на номер сети, а какая – на номер узла.

Примечание. Маска подсети устроена таким образом, что сначала в ней идут строго все **единицы** (номер сети), а затем все **нули** (номер устройства в сети/хост).

Если мы вернемся к примеру с домами и квартирами и наложим на адрес 15648 маску 11100, то получится, что 156 – это номер дома, а 48 – номер квартиры.

А что же такое **адрес сети**?

Важно!!!

Адрес сети – это поразрядная конъюнкция маски и IP-адреса.

Разберем конкретный пример. Возьмем IP-адрес: 192.168.5.73 и маску: 255.255.248.0. Переведем их в двоичную СС и найдем поразрядную конъюнкцию.

Mask	11111111.11111111.11111000.00000000
IP-адрес	11000000.10101000.00000101.01001001
Адрес сети	11000000.10101000.00000000.00000000

Получаем, что адрес сети равен

$$11000000.10101000.00000000.00000000_2 = 192.168.0.0_{10}$$

Для этого же примера найдем номер компьютера в сети. Напомним себе, что он находится в IP-адресе там, где в маске сети находятся нули. Получаем, что номер компьютера в сети равен:

$$10101001001_2 = 1353_{10}$$

1.3 Модуль `ipaddress`

В *Python* существует модуль **`ipaddress`**, с помощью которого можно программно решать задание №13.

Подключение библиотеки происходит следующим образом:

```
from ipaddress import *
```

Рассмотрим основные функции этого модуля и другие методы для успешной работы с ним.

1.3.1 Функция `ip_address()`

С помощью функции `ip_address()` можно создать объект IP-адреса. В качестве аргумента функции мы указываем строку, в которой записано значение IP-адреса:

```
from ipaddress import *
```

```
ip = ip_address('186.153.196.175')
```

```
print(ip)
```

```
# Вывод: 186.153.196.175
```

1.3.2 Функция `ip_network()`

С помощью функции `ip_network()` можно создать объект сети. Каждая сеть задаётся IP-адресом и маской сети, поэтому в качестве аргумента функции указывается строка, содержащая IP-адрес сети и маску разделённые символом «/».

Важно!!!

С этой функцией может возникнуть проблема: если передать IP-адрес с маской (например, 186.153.192.175/255.255.192.0), *Python* может выдать ошибку, потому что ожидает, что это адрес сети (где все биты хоста должны быть нулевыми).

Чтобы *Python* не ругался, нужно сказать ему: «Не проверяй строго, просто прими этот адрес». Для этого нужно использовать параметр **strict=False**, **strict=0** либо просто **0**.

Что делает этот параметр? Он говорит *Python*: «Даже если в адресе есть ненулевые биты хоста, всё равно обработай его как сеть, просто автоматически обрежь лишнее.»

Пример ошибочного и правильных записей:

```
from ipaddress import *

net = ip_network('186.153.192.175/255.255.192.0')
print(net)
# Ошибка: 186.153.192.175/18 has host bits set

net = ip_network('186.153.192.175/255.255.192.0', strict=False)
print(net)
# Вывод: 186.153.192.0/18

# Дополнительные способы задания переменной net, не меняющие вывода
net = ip_network('186.153.192.175/255.255.192.0', strict=0)
net = ip_network('186.153.192.175/255.255.192.0', 0)
```

С помощью функции **ip_network()** можно перебрать все адреса, которые есть в данной сети. Реализуем это программно:

```
from ipaddress import *

net = ip_network('186.153.192.175/255.255.255.252', 0)
for ip in net:
    print(ip)
# Вывод:
# 186.153.192.172
# 186.153.192.173
# 186.153.192.174
# 186.153.192.175
```

1.3.3 Получение двоичной записи IP-адреса

Каждый IP-адрес (например, 192.168.1.1) можно перевести в обычное десятичное число:

```
from ipaddress import *  
  
ip = ip_address('92.168.1.1')  
new_ip = int(ip)  
print(new_ip)  
# Вывод: 1554514177
```

Затем это число можно преобразовать в двоичный (бинарный) вид с помощью функции bin().

```
from ipaddress import *  
  
ip = ip_address('92.168.1.1')  
new_ip = bin(int(ip))  
print(new_ip)  
# Вывод: 0b1011100101010000000000100000001
```

Чтобы убрать префикс «0b» воспользуемся срезом [2:]:

```
from ipaddress import *  
  
ip = ip_address('92.168.1.1')  
new_ip = bin(int(ip))[2:]  
print(new_ip)  
# Вывод: 1011100101010000000000100000001
```

Как мы знаем, IP-адрес в двоичном виде должен занимать ровно 32 бита (даже если в начале идут нули). Если в IP-адресе содержится меньше бит, необходимо воспользоваться функцией `zfill(32)`:

```
from ipaddress import *

ip = ip_address('92.168.1.1')
new_ip = bin(int(ip))[2:].zfill(32)
print(new_ip)
# Вывод: 01011100101010000000000100000001
```

Получаем итоговую форму перевода IP-адреса в двоичную запись, где *ip* – это исходный IP-адрес:

`bin(int(ip))[2:].zfill(32)`

1.3.4 Принадлежность IP-адреса сети

Чтобы определить принадлежит ли IP-адрес сети можно воспользоваться оператором `in`:

```
from ipaddress import *

if ip_address('145.67.160.123') in ip_network('145.67.160.0/255.255.255.0'):
    print('IP-адрес 145.67.160.123 принадлежит сети.')
else:
    print('IP-адрес 145.67.160.123 не принадлежит сети.')
```

1.3.5 Адрес сети и широковещательный адрес

С помощью модуля **ipaddress** можно проверить является ли IP-адрес адресом сети либо широковещательным адресом. Для этого в модуле существуют два свойства:

- **network_address** – адрес сети;
- **broadcast_address** – широковещательный адрес.

Реализуем проверку в программе:

```
from ipaddress import *

# Задаём сеть и IP для проверки
network = ip_network("192.168.1.0/24")
ip_to_check = ip_address("192.168.1.255")

# Проверяем, является ли IP адресом сети или широковещательным
if ip_to_check == network.network_address:
    print(f"{ip_to_check} - это адрес сети")
elif ip_to_check == network.broadcast_address:
    print(f"{ip_to_check} - это широковещательный адрес")
else:
    print(f"{ip_to_check} - это обычный хост")
```

1.3.6 Другие полезные свойства

Помимо свойств `network_address` и `broadcast_address` для более углубленного изучения можно запомнить следующие:

- `hosts()` – возвращает итератор по доступным адресам хостов (исключая адрес сети и широковещательный);
- `netmask` – маска сети (в формате IP);
- `num_addresses` – общее количество адресов в сети;

Пример работы с этими свойствами:

```
from ipaddress import *

net = ip_network('10.0.0.0/255.255.255.240')

# Вывод всех хостов
for host in net.hosts():
    print(host)

print(f"Количество адресов: {net.num_addresses}")
print(f"Маска: {net.netmask}")
```

2 Разбор прототипов задач

2.1 Поиск байта маски

В терминологии сетей TCP/IP маской сети называется 32-разрядная двоичная последовательность, определяющая, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу самого узла в этой сети. При этом в маске сначала (в старших разрядах) стоят единицы, а затем с некоторого места нули.

Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске. Обычно маска записывается по тем же правилам, что и IP-адрес – в виде четырёх байтов, причём каждый байт записывается в виде десятичного числа.

Пример. Пусть IP-адрес узла равен 231.32.255.131, а маска равна 255.255.240.0. Тогда адрес сети равен 231.32.240.0.

Для узла с IP-адресом 145.67.184.122 адрес сети равен 145.67.160.0. Чему равен третий слева байт маски? Ответ запишите в виде десятичного числа.

Решение руками:

Переведем третий слева байт IP-адреса и адреса сети в двоичную СС. Зная, что адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске, найдем значение маски.

IP-адрес	10111000
Адрес сети	10100000
Mask	11100000

Получаем третий слева байт: $11100000_2 = 224_{10}$

Решение программой:

```
# Импортируем функцию ip_network из модуля ipaddress
from ipaddress import ip_network

# Перебираем все возможные значения маски подсети
for mask in range(1, 32):
    # Создаем объект сети для IP-адреса 145.67.184.122 с текущей маской
    # Параметр strict=False позволяет создать сеть даже если IP не является
    # сетевым адресом
    net = ip_network('145.67.184.122/' + str(mask), 0)
    # Проверка: есть ли строка '145.67.160.0' в строковом представлении сети
    if '145.67.160.0' in str(net):
        # Если условие выполняется, выводим объект сети и маску сети
        print(net, net.netmask)
```

Ответ: 224

2.2 Поиск наибольшего/наименьшего байта маски

В терминологии сетей TCP/IP маска сети – это двоичное число, меньшее 2^{32} ; в маске сначала (в старших разрядах) стоят единицы, а затем с некоторого места нули. Маска определяет, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу самого узла в этой сети. Обычно маска записывается по тем же правилам, что и IP-адрес – в виде четырёх байт, причём каждый байт записывается в виде десятичного числа. Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске.

Например, если IP-адрес узла равен 192.168.5.17, а маска равна 255.255.255.0, то адрес сети равен 192.168.5.0.

Для узла с IP-адресом 140.127.80.1 адрес сети равен 140.127.64.0. Чему равно наибольшее возможное значение третьего слева байта маски? Ответ запишите в виде десятичного числа.

Решение руками:

Переведем третий слева байт IP-адреса и адреса сети в двоичную систему счисления. Зная, что адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске, найдем возможное значение маски.

IP-адрес	01010000
Адрес сети	01000000
Mask	11*00000

Вместо символа «*» может стоять как 0, так и 1. Так как в задаче требуется найти наибольшее значение третьего слева байта маски, то ставим вместо «*» 1.

Получаем значение байта:

$$11100000_2 = 224_{10}$$

Примечание: Если бы в задаче требовалось найти наименьшее значение, то на место «*» стоило поставить 0 и ответ был бы 192.

Решение программой:

```
# Импортируем только нужную функцию из модуля ipaddress
from ipaddress import ip_network

# Перебираем все возможные маски сети
for mask in range(1, 32):
    # Создаем объект сети для IP 140.127.80.1 с текущей маской
    # Параметр strict=False позволяет создать сеть даже если IP не является
    # сетевым адресом
    net = ip_network('140.127.80.1/' + str(mask), strict=False)
    # Проверка: есть ли строка '140.127.64.0' в строковом представлении сети
    if '140.127.64.0' in str(net):
        # Если условие выполняется, выводим объект сети и маску сети
        print(net, net.netmask)
```

Ответ: 224

2.3 Поиск количества значений маски

В терминологии сетей TCP/IP маска сети – это двоичное число, меньшее 2^{32} ; в маске сначала (в старших разрядах) стоят единицы, а затем с некоторого места нули. Маска определяет, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу самого узла в этой сети.

Обычно маска записывается по тем же правилам, что и IP-адрес – в виде четырёх байт, причём каждый байт записывается в виде десятичного числа. Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске.

Например, если IP-адрес узла равен 131.32.255.131, а маска равна 255.255.240.0, то адрес сети равен 131.32.240.0.

Для узла с IP-адресом 1.100.224.252 адрес сети равен 1.100.224.0. Для скольких различных значений маски это возможно?

Решение руками:

Переведем IP-адрес и адрес сети в двоичную СС. Зная, что адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске, найдем возможное значение маски.

IP-адрес	00000001.01100100.11100000.11111100
Адрес сети	00000001.01100100.11100000.00000000
Mask	11111111.11111111.111*****.00000000

На месте символов «*» может стоять как 0, так и 1.

Вспомним, что в маске сначала стоят нули, а затем единицы. Поэтому возможные варианты, которые могут стоять на месте «*****»: 00000, 10000, 11000, 11100, 11110, 11111. Всего их 6 штук.

Решение программой:

```
# Импортируем функцию ip_network из модуля ipaddress для работы с IP-сетями
from ipaddress import ip_network

# Запускаем цикл перебора всех возможных значений масок подсети
for mask in range(1, 32):
    # Создаем объект сети для IP-адреса 1.100.224.252 с текущей маской
    # Параметр strict=False позволяет создать сеть даже если IP не является
    # сетевым адресом
    net = ip_network('1.100.224.252/' + str(mask), strict=False)
    # Выводим информацию о созданной сети:
    # 1. net - объект сети
    # 2. mask - текущее значение маски
    print(net, mask)
```

Ищем в списке сеть 1.100.224.0 и записываем количество масок.

Ответ: 6

2.4 Поиск номера компьютера

В терминологии сетей TCP/IP маска сети – это двоичное число, меньшее 2^{32} ; в маске сначала (в старших разрядах) стоят единицы, а затем с некоторого места нули. Маска определяет, какая часть IP-адреса узла сети относится к адресу сети, а какая — к адресу самого узла в этой сети. Обычно маска записывается по тем же правилам, что и IP-адрес — в виде четырех байт, причем каждый байт записывается в виде десятичного числа. Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске.

Например, если IP-адрес узла равен 131.32.255.131, а маска равна 255.255.240.0, то адрес сети равен 131.32.240.0.

Определите номер компьютера в сети, если маска подсети равна 255.255.255.248 и IP-адрес компьютера в сети 108.23.252.178.

Ответ запишите в десятичной системе счисления.

Решение руками:

Переведем IP-адрес и маску сети в двоичную СС.

IP-адрес	01101100.00010111.11111100.10110010
Mask	11111111.11111111.11111111.11111000

Так как в маске последние 3 бита – нули, то номер компьютера в сети равен:

$$010_2 = 2_{10}$$

Решение программой:

```
# Импортируем все функции из модуля ipaddress
from ipaddress import *

# Создаем объект сети для IP 108.23.252.178 с маской 255.255.255.248
# Параметр strict=False позволяет создать сеть даже если IP не является
# сетевым адресом
net = ip_network('108.23.252.178/255.255.255.248', strict=False)
# Инициализируем счетчик для нумерации адресов в сети
с = 0

# Перебираем все IP-адреса в созданной сети
for i in net:
    # Проверка: есть ли строка '108.23.252.178' в строковом представлении текущего
    if '108.23.252.178' in str(i):
        # Если совпадение найдено, выводим:
        # 1. Порядковый номер адреса в сети
        # 2. Сам IP-адрес
        print(с, i)
    # Увеличиваем счетчик на каждой итерации
    с += 1
```

Ответ: 2

2.5 Поиск количества адресов в сети

В терминологии сетей TCP/IP маской сети называется двоичное число, определяющее, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу самого узла в этой сети. При этом в маске сначала (в старших разрядах) стоят единицы, а затем с некоторого места – нули. Обычно маска записывается по тем же правилам, что и IP-адрес, – в виде четырёх байтов, причём каждый байт записывается в виде десятичного числа. Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске.

Например, если IP-адрес узла равен 231.32.255.131, а маска равна 255.255.240.0, то адрес сети равен 231.32.240.0.

Для узла с IP-адресом 108.212.62.226 адрес сети равен 108.212.62.224. Чему равно наибольшее количество возможных адресов в этой сети?

Примечание. Адрес сети и широковещательный адрес не учитывать при подсчёте.

Решение руками:

Переведем четвертый слева байт IP-адреса и адреса сети в двоичную систему счисления. Зная, что адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске, найдем возможное значение маски.

IP-адрес	11100010
Адрес сети	11100000
Mask	111***00

где вместо символа «*» может стоять как 0, так и 1. Так как по условию задачи требуется наибольшее количество адресов, то единиц должно быть по минимуму, значит, ставим вместо «x» нули. Таким образом, количество нулей в последнем байте маски равняется 5.

Следовательно, наибольшее количество возможных адресов в этой сети равняется $2^5 = 32$.

Но ещё нужно учесть, что адрес сети и широковещательный адрес входит в это количество. Значит, итоговый ответ: $32 - 2 = 30$.

Решение программой:

```
# Импортируем все функции из модуля ipaddress
from ipaddress import *

# Перебираем все возможные значения масок подсети от 1 до 31 бита
for mask in range(1, 32):
    # Создаем объект сети для IP 108.212.62.226 с текущей маской
    # Параметр strict=False позволяет создать сеть даже если IP не является
    # сетевым адресом
    net = ip_network('108.212.62.226/' + str(mask), strict=False)
    # Проверка: есть ли строка '108.212.62.224' в описании сети
    if '108.212.62.224' in str(net):
        # Если условие выполняется, выводим:
        # 1. Объект сети
        # 2. Количество доступных хостов (исключая адрес сети и широковещательный)
        print(net, len(list(net.hosts())))
```

В выводе получаем список:

108.212.62.224/27 30

108.212.62.224/28 14

108.212.62.224/29 6

108.212.62.224/30 2

Так как по условию необходимо найти наибольшее количество адресов, то ответом будет являться число 30.

Ответ: 30

2.6 Кратность единиц IP-адреса

В терминологии сетей TCP/IP маска сети – это двоичное число, меньшее 2^{32} ; в маске сначала (в старших разрядах) стоят единицы, а затем с некоторого места нули. Маска определяет, какая часть IP-адреса узла сети относится к адресу сети, а какая – к адресу самого узла в этой сети.

Обычно маска записывается по тем же правилам, что и IP-адрес – в виде четырёх байт, причём каждый байт записывается в виде десятичного числа. Адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске.

Например, если IP-адрес узла равен 131.32.255.131, а маска равна 255.255.240.0, то адрес сети равен 131.32.240.0.

Сеть задана IP-адресом 58.27.142.237 и сетевой маской 255.255.255.240.

Сколько в этой сети IP-адресов, для которых количество единиц в двоичной записи IP-адреса кратно 3?

В ответе укажите только число.

Решение руками:

Переведем IP-адрес и маску сети в двоичную систему счисления. Зная, что адрес сети получается в результате применения поразрядной конъюнкции к заданному IP-адресу узла и маске, найдем возможное значение адреса сети.

IP-адрес	00111010.00011011.10001110.11101101
Mask	11111111.11111111.11111111.11110000
Адрес сети	00111010.00011011.10001110.1110****

На месте символов «****» могут стоять как 1, так и 0. Так как на этой стадии в адресе сети уже есть 15 единиц, то есть количество кратно 3, то нам нужно добавить либо 0, либо 3 единицы.

Количество способов поставить 0 единиц на 4 места (то есть поставить все нули)

– 1. Количество способов поставить 3 единицы на 4 позиции: $C_4^3 = \frac{4!}{3! \cdot 1!} = 4$. Общее количество IP-адресов: $1 + 4 = 5$.

Решение программой:

```
# Импортируем все функции из модуля ipaddress
from ipaddress import *

# Инициализируем счетчик для подсчета подходящих IP-адресов
count = 0

# Создаем объект сети для IP 58.27.142.237 с маской 255.255.255.240
# Параметр strict=False позволяет создать сеть даже если IP не является
# сетевым адресом
net = ip_network('58.27.142.237/255.255.255.240', 0)

# Перебираем все IP-адреса в созданной сети
for i in net:
    # Преобразуем IP-адрес в целое число, затем в двоичную строку (без '0b')
    # Считаем количество единиц в двоичном представлении
    # Проверяем, кратно ли количество единиц трем
    if bin(int(i))[2:].count('1') % 3 == 0:
        # Если условие выполняется, увеличиваем счетчик
        count += 1

# Выводим общее количество IP-адресов, удовлетворяющих условию
print(count)
```

Ответ: 5

3 Заключение

Прорезюмируем, что Вы теперь знаете и умеете:

- Знаете как формируется IP-адрес
- Знаете, что такое адрес сети и маска сети
- Знаете, где определяется номер устройства в сети
- Знаете как работать с модулем ipaddress

Удачи на экзамене! Ваша команда Школково по Информатике.

*Подробнее изучить работу с базами данных вы можете на вебинарах
«Школково»*

А также подписавшись на наши социальные сети:

